

Deep Learning (CAI-461)

Objectives:

- Deep Belief networks
 - Structure of DBN
 - Learning in DBNs
-

Deep Belief Networks:

Geoffrey Hinton et al. introduced deep belief networks (DBNs) in 2006. These deep learning algorithms consist of latent variables and use them to learn underlying patterns within the data. The underlying nodes are linked as a directed acyclic graph (DAG), giving the network generative and discriminative qualities. DBNs work similarly to traditional multi-layer perceptron (MLPs) and offer certain benefits over them, including faster training and better weight initialization.

DBNs consist of various layers of neurons, each connected to the neuron of the subsequent layer. The overall architecture is similar to an MLP, but only the layers are connected, and there are no intra-layer connections.

Each layer can be considered a separate model as it trains independently on the output of the previous one. This way, a DBN is a stack of networks, each of which has learned different features and traits from the raw data.

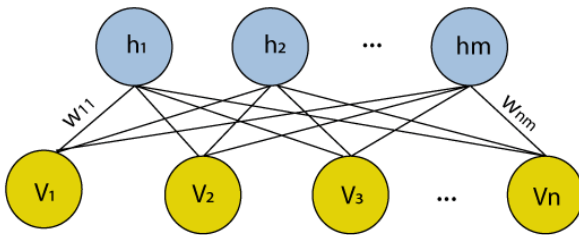
The complex layer-wise neural architecture allows DBNs to work on both supervised and unsupervised problems. Moreover, their ability to comprehend complex underlying data patterns makes them excellent for generative applications. Let's look at the architecture in detail.

Deep Belief Network (DBN) Structure

A deep belief network is a stack of multiple Restricted Boltzmann Machine (RBM) structures that form the foundation of deep architectures. Each of these RBMs consists of a visible layer and a hidden layer. The visible layer accepts the input from the previous layer, while the hidden layer stores the processed output. For a better understanding, we need to understand the RBM first.

Restricted Boltzmann Machines

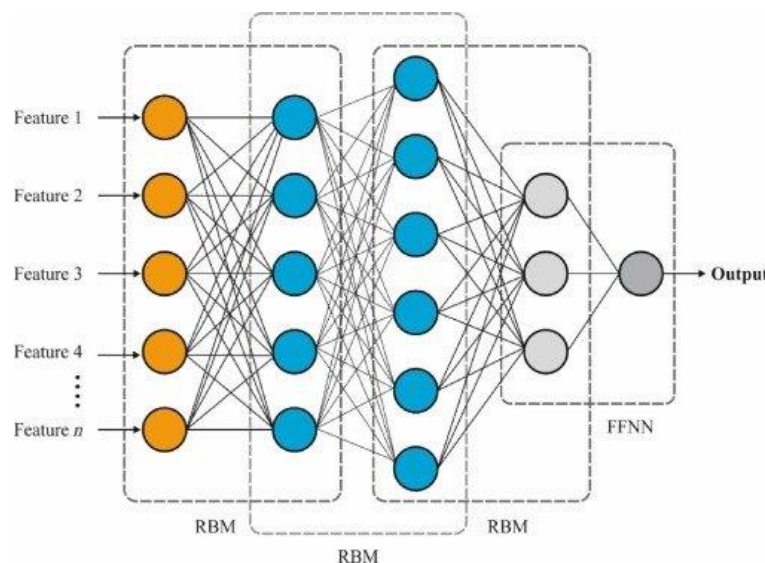
A Restricted Boltzmann Machine is a two-layer probabilistic neural network. Its concept was first introduced in 1986 as the Harmonium model, but the term RBM was not used until the mid-2000s. Its first layer (visible layer) interacts with the raw data, and the second (hidden layer) learns high-level features from the first one.



They are ‘*Restricted*’ since the connections only exist between neurons in subsequent layers. RBMs have performed excellently in forming data representations and gained popularity in tasks such as collaborative filtering.

Structure of DBNs

A Deep Belief Network extends the RBM functionality by creating overlapping stacks of the model. The stacks overlap since the hidden layer of one model is the visible layer of the next one. These RBMs are trained independently, and the overall architecture forms the deep belief network.



Each RBM within the DBN is an energy-based model. This means that it uses an energy value to define the relationship between its hidden and visible units. The energy value is calculated using a linear combination of the unit activations, pairwise combinations of the activations, and any biases associated with the units.

A lower energy means a higher probability of association between the units. By minimizing the energy value for the overall network, the RBM constructs a plausible representation of the original data.

DBN Training

The training of a deep belief network consists of a pre-training phase and then task-specific fine-tuning. The two methodologies are a hybrid of unsupervised and supervised learning approaches.

Pre-Training

The pre-training phase aims to initialize the DBN so that the trained weights directly represent the input data. We perform this pre-training phase with unsupervised learning and train each RBM module independently as a feature detector.

The ensemble's first layer (often called the input layer or bottom layer) interacts directly with the raw data, learning its features and creating a latent representation during the process. Each subsequent layer is then trained so the first's output becomes the next's input. This greedy layer-wise learning allows for efficient feature learning.

The RBM training is conducted using the Contrastive Divergence algorithm. The training comprises a positive phase and a negative phase. The positive phase samples the activation of the visible layer to calculate probabilities for the activation of the hidden units and vice versa for the negative phase.

We iterate this process multiple times, covering various data samples and updating weights after each pass. Finally, the last layer (output layer) outputs the network's prediction. Formula of Contrastive Divergence is given below

$$p(v_i = 1 | h) = \frac{1}{1 + e^{-(a_i + \sum_j h_j w_{ij})}} = \tilde{O}(a_i + \sum_j h_j w_{ij})$$

Fine-Tuning

Once we've completed the pre-training and initialized the model weights, it is further fine-tuned for downstream tasks. We perform fine-tuning using labeled data and a supervised learning algorithm like backpropagation. This way, the model can be trained for various tasks, including classification or regression, and the initialized weights result in faster training and better results.

Benefits of DBNs

Deep belief networks use probabilistic modeling and a supervised learning approach to offer certain benefits over conventional neural networks. These include:

- Ability to handle large data using hidden units to extract underlying correlations.
- Faster training and better results.
- Achieving global minima due to better weights initialization.

DBN Applications

Deep Belief Networks work similarly to deep neural networks and can handle large datasets and various data types. Therefore, they are great for tasks like:

- Image Classification

- Text Classification.
- Image Generation.
- Speech Recognition.

Deep belief networks did not gain much popularity despite demonstrating hybrid capabilities. Deep belief networks played a vital role in the growth and evolution of deep learning, but are not so common today.

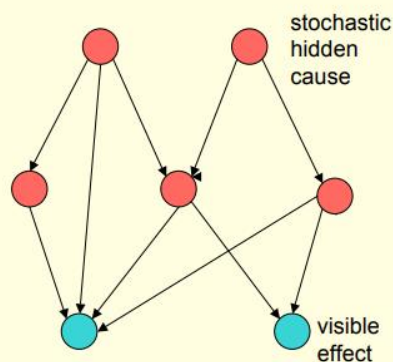
What is wrong with back-propagation?

- It requires labeled training data.
 - Almost all data is unlabeled.
- The learning time does not scale well
 - It is very slow in networks with multiple hidden layers.
- It can get stuck in poor local optima.

Overcoming the limitations of backpropagation

- Keep the efficiency and simplicity of using a gradient method for adjusting the weights, but use it for modeling the structure of the sensory input.
 - Adjust the weights to maximize the probability that a generative model would have produced the sensory input.
 - Learn $p(\text{image})$ not $p(\text{label} | \text{image})$
- If you want to do computer vision, first learn computer graphics
- What kind of generative model should we learn?

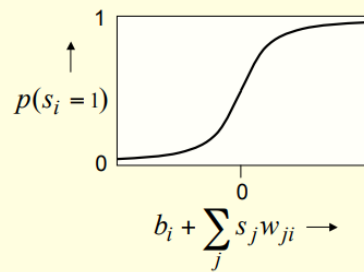
- A belief net is a directed acyclic graph composed of stochastic variables.
- We get to observe some of the variables and we would like to solve two problems:
- **The inference problem:** Infer the states of the unobserved variables.
- **The learning problem:** Adjust the interactions between variables to make the network more likely to generate the observed data.



We will use nets composed of layers of stochastic binary variables with weighted connections. Later, we will generalize to other types of variable.

Stochastic binary units (Bernoulli variables)

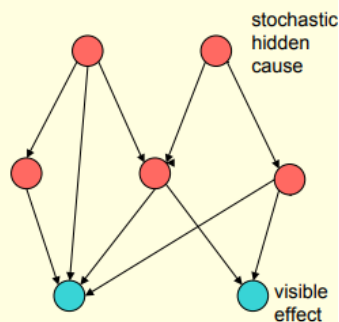
- These have a state of 1 or 0.
- The probability of turning on is determined by the weighted input from other units (plus a bias)



$$p(s_i = 1) = \frac{1}{1 + \exp(-b_i - \sum_j s_j w_{ji})}$$

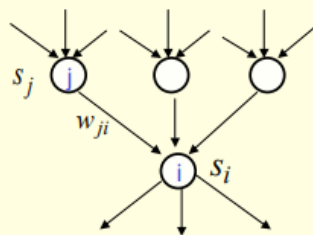
Learning Deep Belief Nets

- It is easy to generate an unbiased example at the leaf nodes, so we can see what kinds of data the network believes in.
- It is hard to infer the posterior distribution over all possible configurations of hidden causes.
- It is hard to even get a sample from the posterior.
- So how can we learn deep belief nets that have millions of parameters?



The learning rule for sigmoid belief nets

- Learning is easy if we can get an unbiased sample from the posterior distribution over hidden states given the observed data.



- For each unit, maximize the log probability that its binary state in the sample from the posterior would be generated by the sampled binary states of its parents.

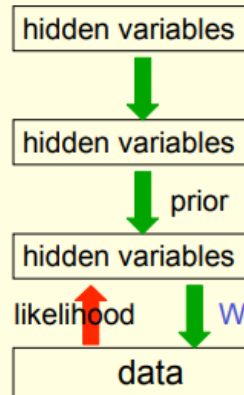
$$p_i = p(s_i = 1) = \frac{1}{1 + \exp(-\sum_j s_j w_{ji})}$$

$$\Delta w_{ji} = \epsilon s_j (s_i - p_i)$$

↑
learning rate

Why it is usually very hard to learn sigmoid belief nets one layer at a time

- To learn W , we need the posterior distribution in the first hidden layer.
- Problem 1:** The posterior is typically complicated because of “explaining away”.
- Problem 2:** The posterior depends on the prior as well as the likelihood.
 - So to learn W , we need to know the weights in higher layers, even if we are only approximating the posterior. All the weights interact.
- Problem 3:** We need to integrate over all possible configurations of the higher variables to get the prior for first hidden layer. Yuk!



Two types of generative neural network

- If we connect binary stochastic neurons in a directed acyclic graph we get a Sigmoid Belief Net (Radford Neal 1992).
- If we connect binary stochastic neurons using symmetric connections we get a Boltzmann Machine (Hinton & Sejnowski, 1983).
 - If we restrict the connectivity in a special way, it is easy to learn a Boltzmann machine.

The Energy of a joint configuration

(ignoring terms to do with biases)

$$E(v, h) = - \sum_{i,j} v_i h_j w_{ij}$$

binary state of visible unit i binary state of hidden unit j
 Energy with configuration v on the visible units and h on the hidden units weight between units i and j

$$-\frac{\partial E(v, h)}{\partial w_{ij}} = v_i h_j$$

Weights → Energies → Probabilities

- Each possible joint configuration of the visible and hidden units has an energy
 - The energy is determined by the weights and biases (as in a Hopfield net).
- The energy of a joint configuration of the visible and hidden units determines its probability:

$$p(v, h) \propto e^{-E(v, h)}$$

- The probability of a configuration over the visible units is found by summing the probabilities of all the joint configurations that contain it.

Using energies to define probabilities

- The probability of a joint configuration over both visible and hidden units depends on the energy of that joint configuration compared with the energy of all other joint configurations.

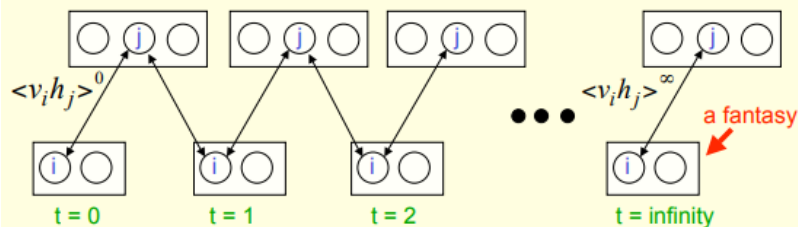
$$p(v, h) = \frac{e^{-E(v, h)}}{\sum_{u, g} e^{-E(u, g)}}$$

partition function

- The probability of a configuration of the visible units is the sum of the probabilities of all the joint configurations that contain it.

$$p(v) = \frac{\sum_h e^{-E(v, h)}}{\sum_{u, g} e^{-E(u, g)}}$$

A picture of the maximum likelihood learning algorithm for an RBM



Start with a training vector on the visible units.

Then alternate between updating all the hidden units in parallel and updating all the visible units in parallel.

$$\frac{\partial \log p(v)}{\partial w_{ij}} = \langle v_i h_j \rangle^0 - \langle v_i h_j \rangle^\infty$$